INTRODUCTION TO AI AGENTS IN 20 TERMS.

20 TERMS WITH CODE — BEGINNER-FRIENDLY.



Chandra Sekhar
AI Engineer & Educator

All 20 Terms

AI Agent concepts

1	Agent
2	Environment
3	Perception
4	State
5	Memory
6	Large Language Model (LLM)
7	Reflex Agent
8	Knowledge Base
9	CoT (Chain of Thought)
10	ReAct
11	Tools
12	Action
13	Planning
14	Orchestration
15	Handoffs
16	Multi-Agent System
17	Swarm
18	Agent Debate
19	Evaluation
20	Learning Loop

Let's dive deep!

1 Agent

An agent is an LLM-driven decision maker that can choose what to do next (call tools, ask follow-ups, compute, search) instead of only answering once like a normal chat.

```
 from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor,
create_react_agent
from langchain import hub

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools=[],
prompt=prompt)
AgentExecutor(agent=agent, tools=
[]).invoke({"input": "Plan my day in 3 steps"})
```

Follow Chandra for More



Repost to help others in your network

2 Environment

The environment is the world the agent acts in: browser, Slack, DB, filesystem, CRM, APIs. In LangChain, it's usually tools + external state your app provides.

```
 env = {"user_role": "admin", "timezone":
"Asia/Kolkata"}
# tools will read/act on environment (APIs, DB,
etc.)
```

3 Perception

Perception means reading the environment (user message, DB results, website HTML) and converting it into something the agent can reason over.

```
 # Perception = taking raw input and structuring
it
user_msg = "Need a refund for order #123"
perceived = {"intent": "refund", "order_id": 123}
```

Follow Chandra for More



Repost to help others in your network

4 State

State is the agent/app's current working context: what step it's on, intermediate results, chosen plan, tool outputs. In LangGraph, state is first-class.

```
 state = {"step": "collect_order_id", "order_id":
None, "notes": []}
```

5 Memory

Memory stores past conversation/context so the agent can stay consistent (preferences, past decisions). LangChain offers memory objects.

```
 from langchain.memory import
ConversationBufferMemory


memory =
ConversationBufferMemory(return_messages=True)
# memory saves conversation history for future
agent calls
```



Follow Chandra for More **Repost to help others in your network**

6 Large Language Model (LLM)

The LLM is the reasoning + generation engine. In LangChain, it's usually ChatOpenAI() or another provider.

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0.2)
```

7 Reflex Agent

A reflex agent does not plan deeply. It follows simple rules: if condition, do action. In LangChain, this might be app logic before calling LLM.

```
def reflex_route(msg: str):
    if "password" in msg.lower():
        return "send_password_reset_link"
    return "ask_llm"

print(reflex_route("I forgot my password"))
```

Follow Chandra for More



Repost to help others in your network

8 Knowledge Base

A knowledge base is stored knowledge (docs/FAQs/wiki) that the agent can retrieve from. In LangChain, typically a vector store retriever (RAG).

```
 # concept-only: retriever represents your KB access
retriever = ...  # e.g., FAISS/Chroma retriever
docs = retriever.invoke("What is the leave policy?")
```

9 Tools

Tools are functions the agent can call to do real work: search, DB queries, calculators, sending emails.

```
 from langchain_core.tools import tool

@tool
def get_order_status(order_id: int) -> str:
    return f"Order {order_id} is in transit"
```



Follow Chandra for More

Repost to help others in your network

10 CoT (Chain of Thought)

You want the model to reason reliably but usually don't need to print chain-of-thought. Enforce structured steps using prompts/outputs.

```
 from langchain_core.prompts import
ChatPromptTemplate
from langchain_core.output_parsers import
StrOutputParser
from langchain_openai import ChatOpenAI


prompt = ChatPromptTemplate.from_template(
  "Solve step-by-step internally, but output only
the final answer.\nQ: {q}"
)
chain = prompt | ChatOpenAI(model="gpt-4o-mini") |
StrOutputParser()
chain.invoke({"q": "What is 19*23?"})
```

Follow Chandra for More



Repost to help others in your network

11 ReAct

ReAct = Reason + Act in a loop: model thinks → calls tool → observes output → continues. Classic tool-using agent pattern in LangChain.

```python
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    return a + b

tools = [add]
# Then create_react_agent(..., tools=tools) like earlier
```

Follow Chandra for More



Repost to help others in your network

12 Action

An action is the actual execution step: calling a tool, storing info, sending message. In LangChain: a tool call is an action.

```
 status = get_order_status.invoke({"order_id":
123})
print(status)
```

13 Handoffs

A handoff is when one agent finishes and passes context to another specialist agent. In LangGraph: node A → node B with shared state.

```
 # state passes between nodes: researcher ->
writer
state = {"topic": "AI Agents", "research_notes":
"...."}
# writer node reads research_notes and produces
draft
```



Follow Chandra for More **Repost to help others in your network**

14 Planning

Planning means the agent first decides a sequence of steps before acting. In LangChain, common approach is plan generation as a chain.

```
 from langchain_core.prompts import
ChatPromptTemplate
from langchain_openai import ChatOpenAI

planner = ChatPromptTemplate.from_template(
  "Create a 3-step plan to: {task}. Output as
numbered list."
) | ChatOpenAI(model="gpt-4o-mini")

planner.invoke({"task": "prepare a client demo for
tomorrow"})
```

Follow Chandra for More



Repost to help others in your network

15 Orchestration

Orchestration = coordinating multiple chains/tools/agents into a pipeline. LangGraph is great for this (state machine / graph).

```
from langgraph.graph import StateGraph

# concept: build a graph with nodes (steps) and edges (flow)
graph = StateGraph(dict)
# graph.add_node("step1", fn);
graph.add_edge("step1","step2"); ...
```



Follow Chandra for More

Repost to help others in your network

16 Multi-Agent System

A multi-agent system is multiple agents collaborating: one researches, one writes, one verifies. Implementation is usually orchestration + handoffs.

```
 researcher_llm = ChatOpenAI(model="gpt-4o-mini")
writer_llm = ChatOpenAI(model="gpt-4o-mini")
reviewer_llm = ChatOpenAI(model="gpt-4o-mini")
```

17 Swarm

A swarm is many small agents producing outputs independently, then you aggregate (vote/rank/merge). Simple pattern: run N agents → pick best.

```
 ideas = ["Idea A", "Idea B", "Idea C"]   # from
multiple agents
best = max(ideas, key=len)   # toy scoring; real:
evaluator agent scores
```

Follow Chandra for More



Repost to help others in your network

18 Agent Debate

Debate = two agents argue opposite sides, then a judge agent decides.

```
 pro = "Use Redis: faster caching."
con = "Avoid Redis: adds ops complexity."
judge_prompt = f"Decide best choice:\nPRO:
{pro}\nCON: {con}\nFinal:"
# judge_llm.invoke(judge_prompt)
```

19 Learning Loop

A learning loop means the agent improves over time using feedback: store failures, adjust prompts, update KB, add rules/tools. Simple pattern: save feedback → change next run.

```
 feedback_log = []
feedback_log.append({"bad": "too long", "fix": "be
concise next time"})
```



Follow Chandra for More **Repost to help others in your network**

20 Evaluation

Evaluation checks agent quality: correctness, completeness, grounding, format, hallucination risk. You can build an LLM-as-judge evaluator chain.

```
 from langchain_core.prompts import
ChatPromptTemplate
from langchain_openai import ChatOpenAI

eval_chain = ChatPromptTemplate.from_template(
  "Score the answer 0-10 for correctness.\nQ:
{q}\nA:{a}\nScore:"
) | ChatOpenAI(model="gpt-4o-mini")

eval_chain.invoke({"q": "2+2?", "a": "4"})
```

Follow Chandra for More



Repost to help others in your network

Want more content like this?



Tap that follow button and stay in the loop!

Like Comment Share Save